코드베이스 및 실험 결과 총정리 — rsma_ppo

RSMA-AoI 연구 스택

유한 블록길이 MU-MISO 하향링크 시뮬레이터, 제약 보존형 hybrid PPO 에이전트, 4종 참조 optimizer, 그리고 이들을 검증하는 실험 인프라. 지금까지 긍정 결과보다 부정 결과를 더 많이 냈으며, 이는 의도된 설계입니다.

489	17,522	43	17	21	29
테스트 통과	Python 줄	소스 모듈	CLI 스크립트	설정 파일	결과셋

계층 1 · 물리계층

모든 알고리즘이 하나의 PHY 구현을 공유

물리 모델은 한 번만 작성되어 모든 정책·베이스라인·optimizer·진단 도구가 재사용합니다. OMA·SDMA·NOMA·RSMA는 별도 코드 경로가 아니라 같은 모델의 **설정**이며, 이 점이 비교의 타당성을 담보합니다.

```
h_k(t)   = sqrt(beta_k) · g_k(t)                          대규모 × 소규모 페이딩
g_k(t+1) = alpha·g_k(t) + sqrt(1-alpha²)·z_k(t)           Gauss-Markov, 정상 CN(0,I)
h_hat_k  = sqrt(1-sigma_e²)·h_k + sigma_e·e_k             불완전 CSIT

gamma_c,k = |h_k^H p_c|²                                  common stream SINR
          / (SUM_j |h_k^H p_j|² + sigma_n²)
gamma_p,k = |h_k^H p_k|²                                  private SINR, SIC 이후
          / (SUM_j≠k |h_k^H p_j|² + sigma_n²)

eps = Q( sqrt(n)·(C(g) - d/n) / sqrt(V(g)) )              FBL 정규근사 블록오류율
P_succ,k = (1 - eps_c,k)·(1 - eps_p,k)                    사용자별 성공확률
```

전환 가능한 모델링 축

축	선택지	기본값
다중접속	`rsma · sdma · oma · noma`	`rsma`
private beam	`mrt · zf · rzf`	`rzf`
common beam	`mrt_sum · principal`	`principal`
링크 모델	`fbl · shannon · fixed`	`fbl`
rate split	`simplex_scaled · per_user_fraction`	`simplex_scaled`
경로손실	`homogeneous · explicit_near_far · distance_pathloss`	`homogeneous`
β 정규화	`none · mean_one · cell_edge_reference`	`mean_one`
CSIT 오차	`relative · absolute · normalized`	`relative`
AoI 가중치	`explicit · equal · inverse_snr · class_priority`	`explicit`
피드백	`sampled · expected`	`sampled`

검증된 동치성

슬롯 단위 **transmit()** 뒤에 벡터화 배치 평가기가 있고, transmit은 N=1로 이를 호출합니다. 두 경로가 **RSMA·SDMA·OMA 세 스킴에서 비트 단위로 동일함**을 테스트로 고정했기 때문에, 수백만 개 할당을 탐색하는 optimizer가 환경이 실제 시뮬레이션하는 것과 어긋날 수 없습니다. NOMA는 조용히 잘못된 경로를 쓰는 대신 예외를 발생시킵니다.

계층 2 · 환경

Generate-at-will AoI를 갖는 Gymnasium 환경

AoI는 성공 시 1로 리셋되고 그 외에는 **min(Δ+1, Δ_max)**로 증가합니다. 환경은 **행동과 무관하게 슬롯당 고정된 개수의 난수를 소비**하므로, 같은 seed에서 모든 알고리즘이 동일한 채널 실현을 봅니다 — 가정이 아니라 테스트로 확인됩니다.

- 7가지 설정 변형에서 **gymnasium.utils.env_checker** 통과
- Dict 행동공간, GAE bootstrap을 위해 terminated와 truncated를 정확히 구분
- 선택적 **계층적 모드**: 에이전트가 스케줄을 정하고 지정된 optimizer가 할당을 풀며, allocator_calls·allocator_evaluations를 보고해 optimizer 비용이 '환경 상호작용' 안에 숨지 않도록 함
- near/middle/far 클래스별 지표를 통합 지표와 함께 산출

계층 3 · 에이전트

제약 보존형 행동 매개화

프로젝트 전체가 의존하는 부분입니다. 샘플 후 clip 방식을 쓰지 않으며, 모든 행동이 구조적으로 실현 가능하고 PPO의 importance ratio가 정확합니다.

구성요소	분포	버퍼 저장	log-prob 채점 대상
스케줄링	Bernoulli×K / subset-categorical / top-m categorical / count-then-subset	raw 샘플 + 보정된 mask	raw 샘플
전력 분할 (K+1)	masked Logistic-Normal	잠재변수 z	unmasked 차원의 Gaussian 로그밀도
율 분할 (K)	masked Logistic-Normal	잠재변수 z	동일
ρ_r 스칼라	Beta 또는 squashed Gaussian	값 또는 잠재변수	Beta logpdf 또는 Gaussian − sigmoid Jacobian

Simplex head는 정책을 **잠재공간 위에** 정의합니다. 환경 행동은 학습 파라미터가 없는 결정론적 변환 **softmax_mask(z)**이므로 PPO는 π(z)를 채점하고, ratio는 저장된 동일 잠재변수와 동일 mask에서 평가됩니다. simplex 위에 직접 밀도를 정의하는 방식은 의도적으로 피했습니다 — softmax는 단사가 아니어서 완전차원 Gaussian의 push-forward 밀도가 정의되지 않기 때문입니다.

핵심 검증 테스트

전체 rollout이 numpy 버퍼를 통과한 뒤, 변경되지 않은 정책으로 저장된 모든 잠재변수를 재채점하면 **max |ratio − 1| < 1e-3**입니다. 잔차는 float32 저장 오차뿐이며, 4가지 스케줄링 head와 계층적 환경 모두에서 성립합니다.

구현된 에이전트와 optimizer

모듈	줄	내용
`agents/ppo.py`	688	GAE, clipped surrogate + value clipping, obs/reward 정규화, target-KL 조기종료, warm start, freeze window, actor/critic 분리 LR, reference-KL penalty, probe 진단

모듈	줄	내용
`agents/distributions.py`	880	스케줄링 head 4종, 스칼라 분포 2종, masked simplex 변환, conditional allocation
`agents/td3.py`	447	twin critic, target smoothing, delayed actor, Polyak target, warm-up — PPO와 동일한 행동 변환 공유
`agents/imitation.py`	502	PHY 평가 기반 모델 선택을 사용하는 oracle behaviour cloning
`agents/allocator_net.py`	283	미분 가능 PHY를 통해 학습하는 objective-aware allocator
`optimizers/rsma_allocator.py`	482	CEM(예산 3종), grid, random shooting, SLSQP, equal, 최근접 캐시
`optimizers/candidates.py`	193	2^K 열거 없이 subset 후보 생성 — K=8까지 확장
`envs/torch_phy.py`	215	미분 가능 PHY 미러, SciPy와 rtol 1e-9로 일치

계층 4 · 평가

공정성 감사를 부가기능이 아닌 인프라로

자원 parity는 설정값 비교가 아니라 **PHY에서 실측**합니다. 감사기는 스킴 간 channel uses, 방사 에너지, 최대 전력, 제공 payload, blocklength, 대역폭을 비교하고, 별도의 geometry 감사가 β_k, 거리, 사용자별 평균 SNR, AoI 가중치, geometry seed를 비교합니다. 두 감사 모두 의도적 위반을 주입해 탐지되는지 회귀 테스트합니다.

모든 실행은 기계 판독 가능한 manifest를 남기며 여기에 gain 부호 규약(**양수 = RSMA 우수**), 확정된 geometry, 비교가 의존하는 6가지 모델링 가정을 기록합니다.

결과 · 실험 캠페인

7개 캠페인과 각각이 확정한 것

캠페인	규모	결과
M0–M2 구축	—	환경, PPO, 베이스라인 8종, 전 마일스톤 실행 가능
스케줄링 붕괴 진단	2,000 states	PPO의 |A|=1이 준최적임을 확인. 구조적 수정 4가지 시도, **전부 반증**
Warm start + 커리큘럼	18 runs	붕괴는 행동적으로 해결, **AoI는 오히려 악화**
RSMA 기여도 ablation	6 arms	해당 운용점에서 common stream은 **순손실**
동질 phase map	5,942만 PHY eval	운용점 3개 확보, RSMA가 **CI 비중첩**으로 승리
Optimizer 검증	2,700 problems	'oracle'을 lower reference로 강등, CEM iteration의 비단조성 발견
이종 phase map	198 cells + 장기 45	수정된 가설 **기각**

긍정 결과

동질 운용점 3곳(K=Nt=2, SNR 15–20 dB, rate 2–3 b/use)에서 CEM allocator를 쓴 RSMA가 장기적으로 두 대안을 모두 이겼으며 신뢰구간이 겹치지 않습니다 — **SDMA 대비 +2.4 ~ +5.1%, OMA 대비 +3.2 ~ +6.8%**. 해당 지점에서 계층적 PPO는 휴리스틱 스케줄러와 정확히 동등했고 end-to-end PPO를 18–23% 앞섰습니다.

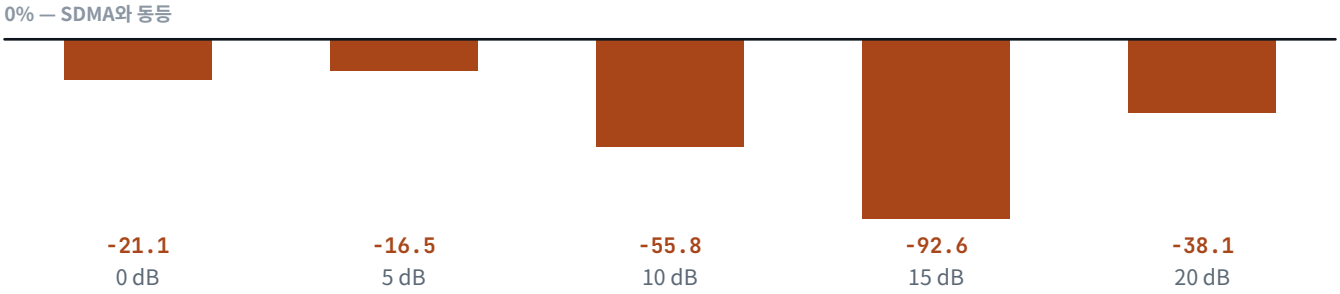
모든 부정 결과를 관통하는 기전

사용자의 패킷은 **(1−ε_c)(1−ε_p)**일 때만 성공하고, common stream은 **모든** 활성 사용자가 복호해야 합니다. 따라서 한 번의 할당 오차가 그들 전부를 동시에 실패시키며, SDMA에서는 한 사용자만 손해를 봅니다. **율 분할은 할당 오차를 평균내지 않고 증폭**합니다. 그래서 allocator가 최적에 가까울 때만 이득이 나고 그렇지 않으면 파국적입니다 — common share를 고정하면 AoI가

11.4–14.8인 반면 같은 스킴에 optimizer를 붙이면 2.1–2.2입니다.

이종 경로손실 가설이 실패한 이유

near/far 격차가 스케줄링과 율 분할을 동시에 중요하게 만들 것으로 기대했습니다. 실제로 스케줄링 가치는 만들어냈고(셀의 85%), 다중사용자가 OMA를 이기게도 했지만(69%), 율 분할 이득은 **파괴**했습니다. common stream은 가장 약한 활성 사용자를 기준으로 해야 하므로, 격차가 크면 common rate가 far 사용자 수준으로 끌려내려가 near 사용자의 용량을 낭비합니다.



근/원거리 격차별 장기 RSMA 이득(SDMA 대비). 무작위 표집한 이종 셀 45개, 전체 에피소드와 샘플링된 ACK 기준. 모든 막대가 동등선 아래이며 평균은 −45.5%, 이득이 조금이라도 있는 셀은 6.7%에 불과합니다.

가장 이전 가능한 발견

운용점 순위를 매기는 데 쓴 myopic one-step 기대 AoI 지표는 **장기 AoI와 무상관**입니다. 상위 14개 셀에서 r = −0.056, 무작위 45개 셀에서 부호 일치 8.9%. +4.7% ~ +8.1%로 선별된 셀들이 장기적으로는 −33% ~ +8.4% 어디에나 떨어집니다. AoI 연구에서 흔한 one-step surrogate 기반 운용점 선정은 목적함수와 접촉하면 무너집니다.

같은 방식으로 잡아낸 두 번째 함정: RSMA가 'OMA를 90% 앞선' 것처럼 보인 지점에서 OMA는 AoI 상한에 고정되어 있었습니다. K=6, Nt=4, rate 3–4 b/use에서 OMA는 12 b/use가 필요한데 단일 사용자 용량은 약 8.6 b/use — 진 것이 아니라 **실행 불가능**합니다. 검증기는 이제 상한의 0.8배 이내인 스킴을 표시합니다.

품질 · 테스트

489개 테스트, xfail·skip 없음

테스트는 줄 수를 채우기 위해서가 아니라 물리·알고리즘 오류를 잡기 위해 작성했습니다. 실제로 여러 테스트가 같은 세션 안에서 먼저 작성한 코드의 진짜 버그를 찾아냈습니다.

영역	실제로 무엇을 검증하는가
PHY	직교 채널에서 손계산 SINR 대조, SINR·blocklength·payload에 대한 FBL 단조성, ZF nulling, 계수 부족 채널 안정성, batch ≡ transmit(3개 스킴)
분포	Beta vs scipy.stats, squashed-Gaussian 밀도의 수치 적분 = 1, 동일 파라미터에서 PPO ratio ≡ 1, masked 잠재변수에 대한 log-prob 불변성
GAE	3-step 손계산 예제, λ=1이 Monte-Carlo return으로 환원, λ=0이 TD 오차로 환원, truncation은 bootstrap하고 termination은 하지 않음
환경	7개 변형에서 Gymnasium checker, 정책 간 동일 채널 시퀀스, seed 재현성, 극단 SNR에서 NaN 없음
Optimizer	모든 allocator의 실현가능성, population과 restart 증가는 도움, CEM iteration 증가는 측정 가능하게 해로움
공정성	스킴 간 parity 성립, 에너지를 의도적으로 2배 준 스킴 탐지, geometry 변조 탐지
Geometry	CSIT absolute 모드가 문서화된 팽창 인공물을 재현, normalized가 이를 제거하면서 원거리 사용자의 상대적 열위는 유지

사용법 · 실행

진입점

검증

```
python -m pytest -q
python scripts/smoke_test.py --config configs/m0_debug.yaml
```

학습

```
python scripts/train_ppo.py --env-config configs/m2_hybrid.yaml \
    --agent-config configs/ppo_default.yaml --seed 0
python scripts/train_td3.py --env-config configs/m2_continuous.yaml \
    --agent-config configs/td3_default.yaml --seed 0
```

비교 및 다중 seed

```
python scripts/run_baselines.py --config configs/m2_hybrid.yaml --seeds 0 1 2
python scripts/run_experiment.py --env-config configs/m2_hybrid.yaml \
    --seeds 0 1 2 3 4 5 6 7 8 9
```

탐색 및 진단

```
python scripts/map_operating_regimes.py --stage coarse --points 150 --workers 12
python scripts/map_heterogeneous_regimes.py --stage coarse --points 200 --workers 12
python scripts/validate_screening.py --csv results/hetero_map/coarse.csv --sample 45
python scripts/diagnose_scheduling.py --config configs/m2_hybrid.yaml \
    --checkpoint runs/m2_seed0/checkpoint.pt
python scripts/verify_optimizer.py --config configs/m2_hybrid.yaml --states 60
```

주의 · 한계

미구현 항목과 신뢰하지 말아야 할 것

- **미구현:** SAC, Jakes/Clarke 도플러 스펙트럼(1차 Gauss-Markov만 있음), 확률적 도착·큐잉(hook만), 행동으로서의 blocklength, recurrent·constrained PPO, full complex precoder 출력, WMMSE/PSO 참조.
- **미실행:** 10-seed 인증과 1M-step 학습 — 어떤 설정도 통과하지 못한 기준에 막혀 있습니다. 기준 미달이 이미 확인된 정책을 인증하는 데 예산을 쓰는 셈이기 때문입니다.
- **myopic screen을 신뢰하지 마십시오.** 이 문서에서 하중을 지탱하는 것은 장기 측정값뿐입니다. phase map은 예측기가 아니라 측정치로서만 유효합니다.
- **긍정 결과는 좁습니다:** 운용점 3개, 각 학습 seed 1개, 그리고 K=Nt=2로 학습할 스케줄링 결정 자체가 없는 지점입니다.
- **계층적 PPO는 현 상태로 배치 불가:** cem_fast 기준 환경 step당 PHY 평가 144회, 동일 상호작용에서 end-to-end PPO의 1.7배 wall clock.

경로 /home/alexalex2/rsma_ppo · Python 3.12, PyTorch 2.14 (CPU), Gymnasium 1.3 · 전체 가정 목록과 유도 과정은 README에 있습니다.